# TALENTRA

# Consolidated Schema Handoff & Benchmark Plan

Milestones 1–10 | Working draft for team review

**Status:** Draft — not an approved or signed schema. Entity lists, relationships, columns and constraints must be reconciled and accepted by the team before implementation.

Purpose: turn the work produced through Milestone 10 into a controlled handoff: identify decisions still needed, define the next implementation steps, and prepare the performance test required by the docket.

# 1. Where the project stands

The previous deliverables cover draft entity inventories, relationship models, column inventories and key/constraint proposals for the first ten milestones. These artifacts are useful inputs, but they do not establish that B23 (signed Phase 1 design) or Phase 2 design approval has been achieved.

| Workstream | Current artifact | Handoff status |
|---|---|---|
| Phase 1: Profile + Jobs & Applications | Milestones 1–6 drafts, ERD, B19 review and decision register | Reconcile entity names, relationships, columns and constraints; obtain team de |
| Phase 2: Administration | Milestones 7–10 draft inventories and constraints | Review role/permission model, audit immutability and session/consent handling |
| Phase 2: Organisation/account | Milestones 7–10 draft inventories and constraints | Review recruiter-to-organisation effective dates and account ownership rules. |
| Phase 2: Gig marketplace | Milestones 7–10 draft inventories and constraints | Define exact assignment-window semantics before implementing overlap protec |

# 2. Freeze a canonical naming and decision register

Before writing a migration, keep one canonical table inventory. Every entity should have one chosen name, one owning module, one purpose, and a decision status. Avoid creating multiple tables for the same concept under different names.

| Field | Required content |
|---|---|
| entity_name | Singular snake_case; one canonical name. |
| module | Profile, Jobs & Applications, Administration, Organisation/account, or Gig marketplace. |
| purpose | One sentence describing the business fact owned by the entity. |
| decision_status | Proposed / Accepted / Rejected / Deferred. |
| owner + date | Person/team accountable for the decision and date recorded. |
| dependencies | Related entities, lookup tables, and external Phase 3 concepts. |

### Required decisions before schema freeze

• Confirm whether every proposed entity is in scope for the relevant milestone; do not silently promote optional ideas to requirements.

• Resolve canonical names and duplicated concepts across the Phase 1 and Phase 2 inventories.

• Confirm lifecycle/status values and whether status history is required as a separate append-only entity.

• Confirm application resubmission, shortlist, offer, rejection, joining and questionnaire-answer behavior.

• Confirm which organisation mapping is effective-dated and how overlapping recruiter assignments are handled.

• Define what counts as an active gig assignment, whether adjacent windows are allowed, and how cancelled/completed assignments affect overlap checks.

• Record delete/update behavior for every foreign key, plus nullable-field rationale and sensitive-data classification.

# 3. Implementation sequence after team approval

| Step | Action | Exit evidence |
|---|---|---|
| A | Team review of decision register and entity inventory | Accepted/rejected decisions recorded; no unresolved naming collisions. |
| B | Create PostgreSQL migration baseline | Versioned migration files; consistent PK/FK naming; repeatable local setup. |
| C | Add keys and constraints | PKs, FKs, UNIQUE, CHECK, indexes and explicit ON DELETE/ON UPDATE behavi |
| D | Add transaction-level rules | Effective-date overlap and gig-window overlap rules tested under concurrent request |
| E | Seed lookup/master data | Stable codes, localized display names, active flags and sort order. |
| F | Build synthetic data generator | Reproducible seed, configurable row counts, referential integrity. |

| Step | Action | Exit evidence |
|---|---|---|
| G | Run validation and benchmark suite | Pass/fail report, query plans, timings, environment and limitations documented. |

Important: do not write a single final production schema by mechanically concatenating the previous PDFs. They are drafts and may contain inconsistent or unapproved entities.

# 4. Docket performance targets

| Dataset | Profiles | Jobs | Applications |
|---|---|---|---|
| Laptop-scale benchmark | 1,000,000 | 100,000 | 5,000,000 |
| Full target (extrapolation target) | 10,000,000 | 1,000,000 | 100,000,000 |

The docket asks for laptop-scale measurement and extrapolation to the full target. Do not claim the full target was tested unless it was actually loaded and benchmarked.

# 5. Synthetic data generation plan

• Use a fixed random seed so the same dataset can be regenerated and compared.

• Generate parent rows before dependent rows; keep all foreign keys valid.

• Use realistic distributions: a mixture of active/inactive profiles, open/closed jobs, and applications spread across jobs and time.

• Include edge cases: profiles with no applications, jobs with no applicants, duplicate natural-key attempts, invalid status transitions, nullability boundaries, and overlapping gig windows.

• Provide separate small smoke-test data and configurable benchmark-scale data.

• Measure generation time and database load time separately from query timings.

### Suggested load order

Lookup/master data → profiles and authentication/account records → organisations and recruiter mappings → jobs → applications → application history/questionnaire/offer/joining dependents → gig entities and assignments → audit/history records.

# 6. Benchmark workload

| Query class | Example workload | Capture |
|---|---|---|
| Point lookup | Fetch a profile by primary key; fetch job by primary key | p50/p95 latency; query plan |
| Filtered list | Search open jobs by status/location/category with pagination | p50/p95; rows scanned; index usage |
| Application list | List applications for one job, ordered by creation time | p50/p95; buffer reads |
| Candidate history | Fetch all applications/status events for one profile | p50/p95; sort and index behavior |
| Aggregates | Count applications by job/status; summary for recruiter organisation | runtime; plan; memory/temp spill if visible |
| Write path | Create application and initial history event in one transaction | throughput; errors; transaction duration |
| Integrity stress | Concurrent overlapping gig assignment attempts | Exactly one conflicting active assignment may commit, if rules |

# 7. Benchmark protocol

• Record laptop CPU, RAM (the team previously stated 16 GB), OS, PostgreSQL version, storage type and configuration.

• Use the same hardware/configuration, dataset seed, query parameters and warm-up procedure for comparable runs.

• Run a small smoke test first, then the 1M / 100k / 5M dataset.

• Warm up the database, run each query class multiple times, and report median plus p95 rather than a single best time.

• Record indexes and EXPLAIN (ANALYZE, BUFFERS) for representative queries; ensure the plan is collected safely on the test dataset.

• Separate cold-start and warmed-cache results if practical; never mix them into one number.

• Document failures, data-generation time, load time, storage consumed and any test limitations.

• Extrapolate carefully: 10× profiles and jobs plus 20× applications will not necessarily mean linear runtime or storage growth.

# 8. Initial acceptance checklist

• Every entity has a documented purpose, module owner and accepted status.

• Every table has a surrogate primary key and consistent key type.

• Foreign keys are database-enforced and specify intended update/delete behavior.

• Natural keys are unique where required but are not used as primary keys by default.

• Business invariants use UNIQUE/CHECK/exclusion constraints or transaction-safe logic as appropriate.

• Money uses fixed precision plus currency code; timestamps are UTC; date-only values use DATE.

• Business tables include the required audit columns or a documented exception.

• Sensitive-data classification and protection decisions are recorded.

• Seed data and synthetic generator are reproducible.

• Benchmark results include environment, dataset sizes, timings, plans and limitations.

• Phase 1 sign-off is explicitly recorded before calling B23 complete.

# 9. Suggested repository layout

```
talentra-db/
  README.md
  docs/
    decisions.md
    entity-inventory.csv
    benchmark-plan.md
    benchmark-results/
  migrations/
    001_create_master_data.sql
    002_create_phase1_profile.sql
    003_create_phase1_jobs_applications.sql
    004_create_phase2_administration.sql
    005_create_phase2_organisation.sql
    006_create_phase2_gig.sql
  seeds/
    master_data.sql
  scripts/
    generate_synthetic_data.py
    load_benchmark_data.sh
  tests/
    schema_constraints.sql
    transaction_rules.sql
    benchmark_queries.sql
```

This layout is a proposal, not a requirement stated verbatim in the docket. Adapt it to the team's chosen repository and tooling.

# 10. Next concrete action

**Do this first:** hold a short schema-freeze review and update the decision register. For each unresolved item, record the selected option and approver. Only after that, create the initial migration and a small smoke-test dataset; do not start with millions of rows.

Document status: working handoff only. This package does not claim migrations, synthetic data, benchmark runs, or team approvals have been completed.